Field Notes

Master path for AI-assisted knowledge work — one git exemplar

Henrik Hellbusch · 2026

How this deck is organized

- **Foundations** — harness thesis, context, mechanism, epistemics, the shift
- **Memory & State** — chat limits, artifacts, memory taxonomy
- **Knowledge Systems** — PKM, pipeline, honest limits
- **Practice** — verify, spar, shoshin, the long game
- **The Workspace** — repo map, toolkit, proactive runtime

Appendix = reading map. Through-line: trust, but verify.

Foundations

Everything is an LLM call

All agent systems → calls to a model provider.

Only difference: the **context** bundled each call — the harness.

Talk: `alex-krentsel-openclaw-deep-dive.md` (Krentsel / OpenClaw architecture)

The harness may matter more than the model

Same model, different harness → very different outcomes.

Terminal Bench: Claude Code ~**#40** · same Opus in other harnesses ~**#1**.

When models are capable enough, the bottleneck is **reliable operation** — not raw IQ.

Library: `alberta-tech-why-devs-obsessed-claude-code.md` · `ryan-lopopolo-harness-engineering.md`

Prompt · context · harness

Layer	Question
Prompt	What to ask?
Context	What to send?
Harness	How does the system run?

Tools, permissions, tests, retries, guardrails — not just tokens in the window.

Library: `tejas-kumar-harnesses-in-ai.md` · Lopopolo · Horthy

The harness

Each model call receives a **bundle of context**:

- Your messages · prior turns · files in scope
- Rules and skills · results from tools

Harness = how that bundle is assembled each turn.

The harness — why structure wins

Clear folders and explicit rules beat a clever one-liner.

Same for code, notes, or a pile of PDFs.

Harness building blocks

Piece	When loaded	Field Notes
AGENTS.md	Every session	Workspace contract
Rules	Always-on	`.cursor/rules/`
Skills	When invoked	`.agents/skills/`

Agent Skills — agentskills.io

- **Skill** = workflow recipe — not an MCP server
- Progressive load: name → `SKILL.md` → linked files
- `/start`, `/spar`, `/review` — portable across Cursor, Pi, Claude Code

Guide: `framework-bootstrap.md`

The context window

- Fixed **token budget** for every session
- Shared by: messages, history, loaded files, tools, rules, skills
- Not infinite — and not all of it is usable

Design question: what earns a place in that budget?

The dumb zone

- Past **~40% context fill**, model reliability drops
- Verbose tool output, many MCPs, long correction chains — all push you there
- **Design for the smart zone:** fresh sessions, intentional scope, RPI

Talk: `dex-horthy-no-vibes-allowed.md`

Context budget — what draws from it

Item	Loaded when
AGENTS.md	Every session
Rules	Always-on
Skills	When invoked
Open files / tools	Harness decides

Every addition is a **tradeoff** — budget it deliberately.

The KV cache — why context has a ceiling

- Context window = a **fixed-size allocation** at session start — not a sliding window
- Allocated from VRAM remaining after weights load → runtime size varies
- **Can't resize mid-session** — earlier tokens evict when it fills

Skip ahead if you don't run local models — the dumb zone is what matters in practice.

Deeper: `docs/ai-engineering/what-a-context-window-actually-is.md`

Software 3.0

Context window = the programming surface.

`AGENTS.md` · rules · skills → not documentation — **they are the code.**

Quality of context determines quality of output.

Talk: `andrej-karpathy-vibe-coding-to-agentic-engineering.md`

Harness problems — the interface layer

- Harness ≠ model only — **editor and product** assemble the bundle
- **Greedy context:** assumes maximum help; fills the window fast
- **Editor bleed:** files in prompt because they were *open*, not in scope
- **Session blend:** unrelated tabs and old chats mixed into one thread

Mitigate: fresh sessions · explicit scope · rules for in/out

Under the hood — the loop stack

What the harness wraps.

Skip to Act II if you just want the practices — come back when model behavior surprises you.

Transformer inference

- **Transformer** — architecture; **weights** learned during training
- **Inference** — forward pass at runtime (**not** training)
- **One step:** tokens in → **P(next token | so far)** → pick one → append

One call → one token — not a whole answer at once.

Transformer inference — the loop

- Append the chosen token → feed the string back → infer again
- A sentence, paragraph, or story = **many** such steps
- **Loop 1** (next slide) is this repetition — Krentsel's first "loopiness"

Sampling (temperature, top-p) usually sits in the **LLM/harness** layer.

Loop 1 — next token

```
tokens in → Transformer → next token → append → …
```

The `…` is the loop above — one token per pass.

Loop 2 — chat

```
You → LLM → reply
 ↑___________|   (conversation)
```

Assistant phase — reactive, turn by turn.

Loop 3 — scoped agent

```
Goal → LLM → tool → result
         ↑___observe___|   (closed control loop)
```

Tools + **act → see outcome → decide next** — not one-shot Q&A.

Same pattern for code, research, and docs.

Loop 4 — autonomous (outer shell)

Broader scope: dynamic tools, self-config, cron/heartbeat.

OpenClaw-class runtimes sit here; Cursor/Pi often sit at Loop 3.

Each layer **wraps** the loops inside — matryoshka.

The nested stack

```
↻ Transformer Inference
  Large-Language Model
  Assistants — ChatGPT, Claude, Gemini
  Scoped agents + tooling — Claude Code, Codex, Cursor
  Autonomous + env. ownership — OpenClaw
```

Each line **wraps** the one above — matryoshka (Krentsel).

Nested stack — inner two layers

Layer	What one "unit" does
Transformer inference	One forward pass → **one** next token
Large-Language Model	**Many** steps — tokenizer, API, sampling

Chat, agents, and harnesses **repeat** these inner calls with more context.

Talk: `alex-krentsel-openclaw-deep-dive.md` (~5:13–7:09)

Statistical, not deterministic

- **Traditional code** — logic you wrote; same input → same output
- **LLM** — statistical model; **P(next token | context)** each step
- **Sampling** — same prompt can yield different answers *(at non-zero temperature)*

Not broken — **plausible**, not guaranteed correct.

Why that changes how you work

- Predicts what **reads well** — not what **is** true
- Mistakes look confident — no crash, no stack trace
- **Harness + verify** — tests, lint, re-read sources

Act IV: **trust, but verify** — same thread.

The shift — what got cheaper

For decades, **implementation** was often the bottleneck.

AI **compresses** it — does not eliminate verification.

Essay: `the-shift.md`

The shift — what matters more

When implementation is cheap, value **often** moves to:

- **Decomposition** — what are we actually building?
- **Verification** — is this correct, safe, sourced?
- **Judgment** — context, ethics, what you sign your name to

Jagged intelligence

Models peak where **RL can verify** — code, math, structured tasks.

Weak where verification is hard — novel judgment, embodied reasoning, ethics.

"What stays human" isn't vague — it's the unverifiable gap.

Talk: `andrej-karpathy-vibe-coding-to-agentic-engineering.md`

Fluency isn't evidence

- AI output looks authoritative — same confidence, right or wrong
- Data-driven posture: *show me the evidence* applies here too
- Treat fluent output like an **unvalidated claim**

Trust, but verify

Trust the speed — drafts and first passes are useful.

Verify before you rely — test, source-check, second look.

Fluent ≠ correct — code, citations, advice.

Sycophancy and agreement bias

- **"Great idea." "You're right."** — praise without evaluation
- Mirrors your frame — confident drafts that *sound* like you
- **Sycophancy:** agreeableness structurally rewarded (RLHF)
- Root: **frictionless** — feels complete without challenge

Essay: `the-shift.md` §6

Push back on agreeableness

- **Treat agreement as a null signal** — validates almost everything
- **No pleasantries** — rule or prompt: skip praise and filler
- **Ask what's wrong** — spar, adversarial review, human pushback

Case study: `frictionless-entity.md`

AI amplifies how you already work

- Tools **amplify** habits and culture — good and bad
- Strong review, handoffs, verification → **compounds faster**
- Shortcuts, rubber-stamping → **compounds faster too**

AI adoption — not bolt-on

Real adoption means **re-evaluating how you work** — not adding a chat box.

Essay: `the-shift.md` — culture amplification

What stays human

When AI drafts faster, you still bring:

- **Clarity** — what is the question? What would change your mind?
- **Verification** — before you stand behind output
- **Judgment** — trade-offs, audience, professional standards
- **Memory** — write down what survived verification

Memory & State

The chat problem — cross-session

New session = blank slate. Prior decisions aren't loaded unless you put them there.

Long chats **compress** earlier context — details become approximations.

Essay: `prompting-and-state.md`

The chat problem — what we skip

"I'll write it up later" rarely happens.

The thread dies; the learning evaporates.

Saved artifacts are memory

> **Files you keep** are memory. **Chat is not.**

Findable · revisable · linkable · diffable.

Saved artifacts — git when ready

A folder works to start.

Git when you want history, branches, and shared review.

Two kinds of memory (Simon Scrapes)

Kind	Question	Field Notes
Operational (L1–4)	What did we decide?	Strong — manual L1–2
Knowledge (L5–6)	What did I learn?	Strong on L5 · `library/`

Don't conflate decision memory with curated learning.

Library: `simon-scrapes-claude-code-memory-systems.md`

Operational memory — L1 and L2

Level 1: always-loaded context (`AGENTS.md`, brief)

Level 2: handoffs — backlog, `whats-next`, session rules

Operational memory — L3+

Semantic search, cron, heartbeat — when *find-it-again* or *autonomous agents* hurt.

Start manual. Automate when friction is real.

Knowledge Systems

Three conversations — usually separate

Conversation	Examples	Field Notes analogue
PKM / second brain	PARA, Zettelkasten, digital garden	capture → distill → express
LLM knowledge base	Karpathy LLM Wiki, Recall	`research/` → `library/`
Agent memory / harness	hooks, memsearch, OpenClaw	rules, skills, handoffs

Three conversations — one git map

This repo **maps all three in git** — one implementation, not the only layout.

Map the pipeline to your stack

Pattern: raw capture → curated wiki → finished output

Paths: `research/` → `library/` → `docs/` · `devops/`

Map the pipeline — schema

Schema matters more than folder names — what goes where, how it's reviewed.

Where this exemplar stops — strong

Strong today: manual **L1–2** · **L5** (`library/`)

Rules, handoffs, backlog · curated library wiki.

Where this exemplar stops — gaps

Not default: automated **L3–4** · **L6** (shared agent brain)

Add when *find-it-again* or *autonomous agents* hurt — not day one.

Capture → synthesize → express

```
research/   →   library/   →   docs/ · devops/
  (raw)           (wiki)        (finished)
```

Research = drawer · **Library** = curated wiki · **Docs** = field notes for others.

Capture pipeline — detail

- **Research** — transcripts, refs, experiment logs
- **Library** — summaries, themes, links
- **Docs / devops** — essays and runnable examples

Karpathy's three layers

LLM Wiki	Field Notes
`raw-sources/`	`research/*/sources/`
`wiki/`	`library/`
`schema/`	`AGENTS.md`, ingest, `/audit`

Karpathy — why schema matters

Raw pile ≠ knowledge base.

Schema — where things go, how they're reviewed — is the third layer.

Library: `karpathy-llm-wiki.md`

Zettelkasten vs essay grain

Zettelkasten: atomic notes, stable IDs, links emerge bottom-up.

Field Notes: synthesis grain — essays, case studies, runnable examples.

Both valid. Essay/case-study grain here — atomic Zettel optional.

When to add a database

Don't replace markdown with SQL by default.

Add vector/graph/Postgres when *find-it-again* or *multi-agent recall* hurts more than curation cost.

Practice

Your first session — three paths

A — code: small task · review diff · test · commit

B — learning: one source · structured notes · **save to a file**

C — chat only: one question — won't compound without B

First session — shared rules

Trust, but verify · small scope · no secrets in prompts

Case study: `docs/ai-engineering/ai-for-unfamiliar-domains.md`

Session habits that compound

Habit	Defends against
Backlog / inbox	"I'll remember later"
Handoff when pausing	context loss between sessions
Re-read source files	in-session compaction

Essay: `session-framework.md`

Session habits (continued)

Habit	Defends against
Checkpoint commits	crash mid-session
Review before merge	fluent wrong output shipping

Add **rules and skills** when friction repeats — not before.

The long game

After 6 months of session discipline:

- `library/` — curated sources you can cite, not chat logs you can't find
- **Rules and skills** — compound; each one makes future sessions faster
- **Review trail** — you know what you verified, when, at which SHA

A pure-chat user has nothing recoverable. You have a workspace.

Sparring and shoshin — the bracket

```
shoshin → draft → spar → revise → share
```

Sparring and shoshin — when

Shoshin (start): read sources; not stale summaries.

Spar (after draft): steel-man counterarguments — thesis, not typos.

Essay: `docs/ai-engineering/sparring-and-shoshin.md`

Zanshin — remaining mind

Zanshin (残心): mind that **remains through** the technique.

The model carries nothing. Framework practices restore what distraction breaks.

Essay: `zanshin.md`

Learning capacity — the full cup

When people are **too full to absorb**, more input spills.

Teams adopting AI: draft throughput can exceed verify capacity.

Essays: `the-full-cup.md` · practitioners guide

The Workspace

What Field Notes is

Public **git notebook** — built from real work over time.

Not a product. Essays · curated library · philosophy · optional infra reference · session rules.

Browse selectively. Fork ideas, ignore rooms that aren't yours.

What's in the repo — core

- `docs/` — essays and case studies
- `library/` — curated sources
- `research/` — raw capture → library
- `devops/` — infra examples *(optional)*

Index: `docs/README.md`

What's in the repo — working layer

- `AGENTS.md` — workspace contract (always loaded)
- `.agents/skills/` · `.cursor/rules/` — skills + rules
- `BACKLOG.md` · `.planning/` — projects in flight

Four ways one notebook serves different readers

1. **Learning** — sources, summaries, experiment logs
2. **Field notes** — write-ups you reuse in other contexts
3. **What people ask for** — links, examples, short guides
4. **Projects in flight** — backlog, handoffs, threads

Four readers — one notebook

Future you · peers · collaborators · tools — different rooms, same artifacts.

Portable toolkit — three layers

Layer	Solves	This workspace
Runtime	where the agent runs; git sync	Paude + Pi
Discipline	verify, spar, session tracking	Zanshin kit
Workspace	accumulated context	this repo

Portable toolkit — one command

Mount runtime + discipline + workspace on a new problem.

Essay: `portable-ai-toolkit.md` *(working draft)*

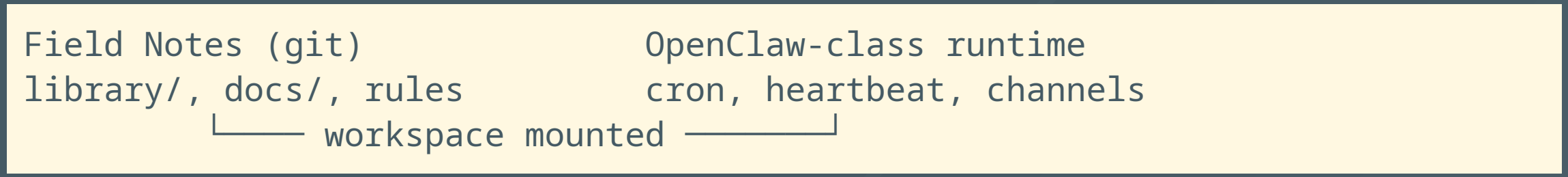
Brain + proactive runtime

Brain — git knowledge you own: `library/`, `docs/`, rules, `/audit`.

Proactive runtime *(OpenClaw-class)* — cron, heartbeat, channels, shell.

Complementary — not either/or.

Brain + runtime — how they connect



Promotion rule: commit to `library/` or `BACKLOG` — not only vector memory.

Library: `alex-krentsel-openclaw-deep-dive.md` *(exploratory)*

What this deck does *not* claim

- One size fits all — **browse selectively**
- Replace your judgment or professional standards

What this deck does *not* claim (continued)

- All repo content is author-reviewed *(much is draft)*
- You need OpenClaw, Paude, or this exact layout

Trust, but verify. The path is yours.

Thanks

Start anywhere — each section stands alone.

1. One place for things worth finding again
2. One bounded AI task — **trust, but verify**
3. One link from the **appendix**

Appendix

Reading map — what to read and why

Pick **one** row. Trust, but verify.

Start here

`the-shift.md`

Why the bottleneck moved — and why AI amplifies the culture you already have.

`ego-ai-and-the-zen-antidote.md`

Why AI can reinforce bad framing — and what to do.

Start here — state

`prompting-and-state.md`

Why a great chat fails across days or weeks.

Start here (continued)

`zanshin.md`

What should survive when the session ends.

`docs/ai-engineering/sparring-and-shoshin.md`

Adversarial review and beginner's mind — no tooling required.

Start here — verification

`frictionless-entity.md`

Root property behind agreement bias and inherited framing.

`docs/ai-engineering/ai-for-unfamiliar-domains.md`

Verification in practice: right answer, wrong first draft.

Learning & teams

`the-full-cup.md`

When people are too full to learn.

`the-full-cup-practitioners-guide.md`

Companion playbook if the essay landed.

Learning & teams (continued)

`session-framework.md`

Session habits — what each defends against and why.

Workflows

`ai-assisted-development-workflows.md`

Multi-session patterns + per-tool context files.

`framework-bootstrap.md`

Load Zanshin posture + AGENTS.md / skills / rules by tool.

Workflows (continued)

`docs/case-studies/building-knowledge-management-with-ai.md`

How backlog, library, and session tools were built.

Memory & PKM

`library/README.md`

Curated talks and articles — learn without starting from zero.

`simon-scrapes-claude-code-memory-systems.md`

Six-level agent memory taxonomy — comparison vocabulary.

Memory & PKM (continued)

`karpathy-llm-wiki.md`

Raw sources vs wiki — why a pile of notes isn't a knowledge base.

`alex-krentsel-openclaw-deep-dive.md`

Agentic loop matryoshka — harness, gateway, proactive runtime.

Harness engineering (AI Engineer)

`ryan-lopopolo-harness-engineering.md`

Harness is the job — docs, lint, reviewer agents; code as build artifact.

`dex-horthy-no-vibes-allowed.md`

Context engineering — stay out of the "dumb zone"; RPI workflow.

Harness engineering (continued)

`tejas-kumar-harnesses-in-ai.md`

First-principles anatomy + demo — verify step; prompt unchanged.

`alberta-tech-why-devs-obsessed-claude-code.md`

Same model, different harness — Terminal Bench ranks the multiplier.

Philosophy & human layer

`docs/case-studies/who-is-speaking.md`

AI writes in your voice — biographical claims, professional identity, opinions attributed to you. Adjacent to sycophancy; a distinct failure mode.

`docs/philosophy/the-dojo-after-the-automation.md`

Who builds the humans capable of directing AI? The capacity to judge and set direction doesn't emerge automatically when execution automates.

Index & toolkit

`portable-ai-toolkit.md`
Runtime + discipline + workspace *(working draft)*.

`docs/README.md` · `devops/` *(optional)*
Master index · runnable infra.